Utrecht University

Replication in Economics

Non-exhaustive tips

Vita Titl

Outline 1

1. Background

2. Repository Structure

3. The Scalars Pattern

4. End-to-End Workflow

What I Saw and/or Did in the Past

- Created v12, v13, _final, and then dates and similar
- Copy pasting and manually adjusting tables (e.g., from outreg2)
- Copy pasting findings – particular numbers – from tables
- No comments, no documentation of which file produces what

This is easy at first, but gets costly when you need to make adjustment to the code (after getting comments from reviewers) or even worse when you need to submit a replication package.

Policies of Journals

Background

Most top journals in economics (and management) require a replication package (top 5s, most top field journals, Management Science, Marketing Science etc.)

Exceptions: confidential administrative data, proprietary datasets; BUT you may still be required to show replication and you post the codes

Details: `https://aeadataeditor.github.io`

Basic requirements

- The code you provide processes raw data, runs all the analysis, and replicates every table and/or figure in the paper.
- You should provide a master script that starts the whole process and the user just waits.
- A readme file needs to provided, giving details of how to replicate the paper
  - The file explains what this replication package does
  - The folder structure
  - List of figures/tables and which code generates it
  - Example config file if needed
  - Requirements (yaml file for Conda environment or requirements.txt for python or a list of Stata packages with versions and sources if not ssc)
  - Memory and runtime requirement

How To Solve This?

- From the beginning, use Git.
- Set the repository well (see later).
- Automate everything possible and have it ready like you were submitting a replication package tomorrow.
- Replicate everything and test whether it works (ideally let co-authors do that).

Outline 2

1. Background
2. Repository Structure
3. The Scalars Pattern
4. End-to-End Workflow

The ideal folder layout

What you need

- One **entry-point**: `code/make.sh`
- An **output** folder that analysis scripts write into
- A **writing** folder with its own `output/` sub-folder that LaTeX reads from
- A sync script: `refresh_output.sh`

Folder tree

```
project/
  code/
    make.sh
    analysis/
    Scalars/
  data/
  dataLocal/
  output/
  writing/paper/
    output/
    main.tex
  refresh_output.sh
```

One entry-point: `make.sh`

A reviewer (or your future self) should be able to run **one command** and reproduce the entire paper:

```
#!/usr/bin/env bash
set -euo pipefail        # stop on any error

# -- Setup ----------------------------------------------------
mkdir -p ../output/logs ../dataLocal
touch ../output/scalars.tex   # initialise scalar store

# -- Analysis (in dependency order) --------------------------
run_stata   analysis/summary_table.do
python      analysis/main_regressions.py

# -- Sync outputs to the writing folder ---------------------
bash ../refresh_output.sh

# -- Compile paper -------------------------------------------
```

`refresh_output.sh` — keeping writing in sync

After each run, `refresh_output.sh` deletes stale copies in `writing/paper/output/` and replaces them with the fresh files from `output/`. Re-running LaTeX then gives the up-to-date paper — **no manual copying**.

```
SRC_DIR="output"
DST_DIR="writing/paper/output"

# Delete all non-tabonly files currently in writing/
find "$DST_DIR" -type f ! -name '*_tabonly.tex' \
  | xargs rm -f

# Re-copy each from output/
while IFS= read -r f; do
    rel="${f#$DST_DIR/}"
    cp "$SRC_DIR/$rel" "$f"
done < <(find "$DST_DIR" -type f)
```

Why the two-folder approach?

Without it

- Manual drag-and-drop after each run
- Forget one table ⇒ wrong numbers in paper
- `writing/` and `output/` silently diverge
- Replicator has no clear path from code to paper

With `refresh_output.sh`

- One command re-links every table and figure
- Fully reproducible: `make.sh` → refresh → `pdflatex` = whole paper
- LaTeX always reads the latest output
- `writing/` can live on Overleaf or a separate git branch

Outline 3

1. Background
2. Repository Structure
3. The Scalars Pattern
4. End-to-End Workflow

What are scalars?

Numbers that appear **inline in the text** of your paper:

> *"The cartel raised prices by* ***\PriceEffect****%."*

> *"We observe* ***\NSampleFirms*** *firms and* ***\NTenders*** *tenders."*

If these are hard-coded, any data change forces manual updates across the paper — and errors creep in.

The scalars pattern

Write every such number into `scalars.tex` as a LaTeX `\newcommand` — **automatically, from your analysis code.**

`scalars.tex` — what it looks like

`output/scalars.tex` accumulates `\newcommand` definitions. The comment records who wrote it and when.

```
% output/scalars.tex  (generated -- do not edit by hand)

\newcommand{\NSampleFirms}{4217}
  % written by jsmith running summary_table.do
  %   on 3 Mar 2026 at 09:14

\newcommand{\PriceEffect}{14.3}
  % written by jsmith running costAnalysis.do
  %   on 3 Mar 2026 at 11:02

\newcommand{\ShareColluding}{.38}
  % written by jsmith running varyingBehaviour.py
  %   on 3 Mar 2026 at 14:55
```


In `main.tex` preamble: `\input{output/scalars.tex}`

Writing scalars — Python

Import `scalar_utils.py` from your `Scalars/` folder. Keys are auto-converted from `snake_case` to `CamelCase` and digits to words (LaTeX macro names cannot contain digits).

```python
import sys; sys.path.append('code')
from Scalars.scalar_utils import save_scalar

fn = '../output/scalars.tex'

# key becomes \NSampleFirms in LaTeX
save_scalar('n_sample_firms',
            df['firm_id'].nunique(), fn)

# inline number from a regression
save_scalar('price_effect',
            round(coef * 100, 1), fn)

# share colluding -> \ShareColluding
```

Writing scalars — Stata

Include `Scalars/scalar_utils.do` at the top of your do-file. Same `scalars.tex` file, shared across all languages.

```
do "code/Scalars/scalar_utils.do"

local fn "../output/scalars.tex"

* Save sample size
count if !missing(price)
save_scalar "NObservations" ‘r(N)’ "‘fn’"

* Save a regression coefficient
reghdfe ln_price cartel, absorb(tender_id)
local coef = round(_b[cartel]*100, .1)
save_scalar "PriceEffect" ‘coef’ "‘fn’"
```

Writing scalars — Julia

`scalar_utils.jl` follows the same convention.

```julia
include("code/Scalars/scalar_utils.jl")

fn = "../output/scalars.tex"

# Save number of auction pairs
save_scalar("NPairs", nrow(df), fn)

# Coefficient with auto-generated key name
# pairwise_fe_coef_one -> \PairwiseFeCoeOneOne
save_scalar("pairwise_fe_coef_one",
            coefs[1], fn)
```

Key point: all three languages write to the *same* `scalars.tex` — no duplication, no divergence.

Using scalars in the paper

Add one line to the `main.tex` preamble, then use any scalar as a macro anywhere in the text.

```
% preamble
\input{output/scalars.tex}

% body -- inline numbers, always up to date:
We study \formatInt{\NSampleFirms} firms across
\formatInt{\NTenders} procurement tenders.

% results paragraph:
Cartel participation raises the final price by
\round{\PriceEffect}\% on average ($p < 0.01$).

% table note referencing a scalar:
The sample covers \NSampleFirms{} unique firms;
\ShareColluding{} of tenders involve a colluding pair.
```

Scalar best practices

Golden rule

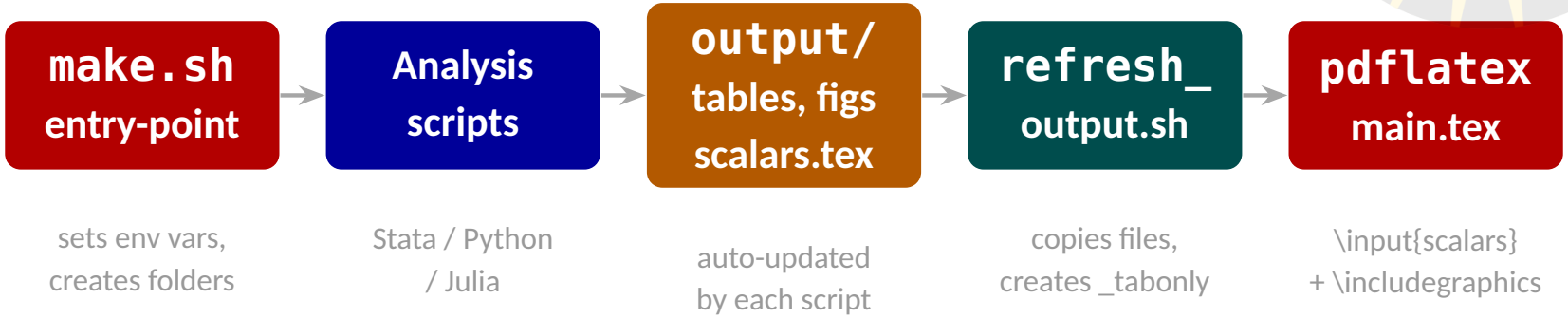
Never hard-code a number in your paper if your code knows it.

- **Key naming:** CamelCase, no digits — e.g. `NTwenty` not `N20` (LaTeX forbids digits in macro names)
- **Audit trail:** comments in `scalars.tex` record the timestamp and script name for every number
- **Cross-language:** Python, Stata, and Julia all append to the same file — no duplication
- **Idempotent writes:** if a key already exists the old line is replaced (`grep` + overwrite), so re-running is safe
- **Read back:** `scalar_utils` also provides `read_scalar` so one script can consume what another produced
- **Timing:** run `refresh_output.sh` *after* `make.sh` so `scalars.tex`

Outline 4

1. Background
2. Repository Structure
3. The Scalars Pattern
4. End-to-End Workflow

From raw data to published paper — one command



`bash code/make.sh` ⇒ updated, compiled paper. **No manual steps.**

Replication package checklist

Before submission:

- ✓ Single entry-point: `bash code/make.sh` reproduces everything
- ✓ `refresh_output.sh` keeps `writing/paper/output/` in sync
- ✓ All inline numbers written via `save_scalar` — zero hard-coded values in the text
- ✓ `scalars.tex` is `\input{}`-ed in `main.tex` preamble
- ✓ README: folder structure, output list per script, runtime & memory
- ✓ Config file for paths (`config.py` or `profile.do`) — no absolute paths in scripts
- ✓ Environment pinned: `conda .yml`, `requirements.txt`, or Stata package list with versions
- ✓ Log files saved per script to `output/logs/`
- ✓ Tested on a clean environment before submission

Random issues to take care of

You need to set seeds Have logs Draftable (make example) – `https://www.draftable.com/compare`